Experiment 6

Student Name: Vishal Saini **UID: 23BCS10163**
Branch: CSE **Section/Group: KRG 3-B**
Semester: 6th **Date of Performance:03/03/2026**
Subject Name: System Design **Subject Code: 23CSH-314**

1. **Aim**: To design and analyze a scalable YouTube-like OTT video streaming platform that allows users to upload, search, and watch videos in multiple resolutions while supporting user interactions such as likes and comments. The system should ensure high availability, low latency, scalability, and efficient video delivery using distributed system architecture.

2. Objective:

- To identify **functional and non-functional requirements** of a large scale video streaming system.
- To design a **High Level Architecture (HLD)** for an OTT platform.
- To understand **video streaming using HLS/DASH manifest files**.
- To design a **microservices based architecture** for video platforms.
- To understand the role of **CDN for fast video delivery**.
- To study how systems like **YouTube scale to millions of users**.

3. Tools:

- **Draw.io / Excalidraw** – System architecture diagrams
- **MySQL / PostgreSQL** – User and metadata storage
- **Object Storage (AWS S3 / Azure Blob)** – Video files
- **CDN (Cloudflare / AWS CloudFront)** – Video distribution
- **Redis** – Caching and session management
- **Load Balancer** – Traffic distribution
- **Microservices architecture** – Service isolation and scalability
- **Video Encoding Tools (FFmpeg)** – Video transcoding
-

4. System Requirements:

A. Functional Requirements:-

1. User should be able to register and login to the platform.
2. User should be able to upload videos.
3. Users should be able to search videos.
4. Users should be able to watch videos in multiple resolutions.

   Example:
   360p, 480p, 720p, 1080p etc.

5. Users should be able to like videos.
6. Users should be able to comment on videos.

B. Non-Functional Requirements:-

1. Scalability

System should support:

- 200M – 500M users
- 100M daily video views

The system must scale horizontally to support millions of concurrent viewers.

2. Availability

Watching videos must be highly available even during server failures.

3. Consistency

Likes and comments must remain consistent across all users.

4. Latency

Video loading latency should be less than 100 ms.

5. Buffering

Video playback should have minimal buffering.

5. High Level Design (HLD):

The system follows a **microservices architecture**.

1. API Gateway

- Authentication
- Authorization
- Request routing
- Rate limiting

2. User Service

- Handles user registration and login
- Stores user data in **User Database**

3. Video Service

- Handles video upload and streaming
- Stores metadata in **Video Metadata DB**

4. Search Service

- Allows users to search videos by name or tags.

5. Like Service

- Manages video likes and counts.

6. Comment Service

- Handles user comments on videos.

7. Blob Storage

- Stores large video files and thumbnails.

8. CDN

- Delivers video content from nearest edge server for faster playback.

6. Video Upload & Processing Pipeline:

Step-1: User uploads video through **Video Upload Service**
Step-2: Video stored in **Blob Storage (S3)**
Step-3: Video encoder converts video into multiple resolutions

Example:

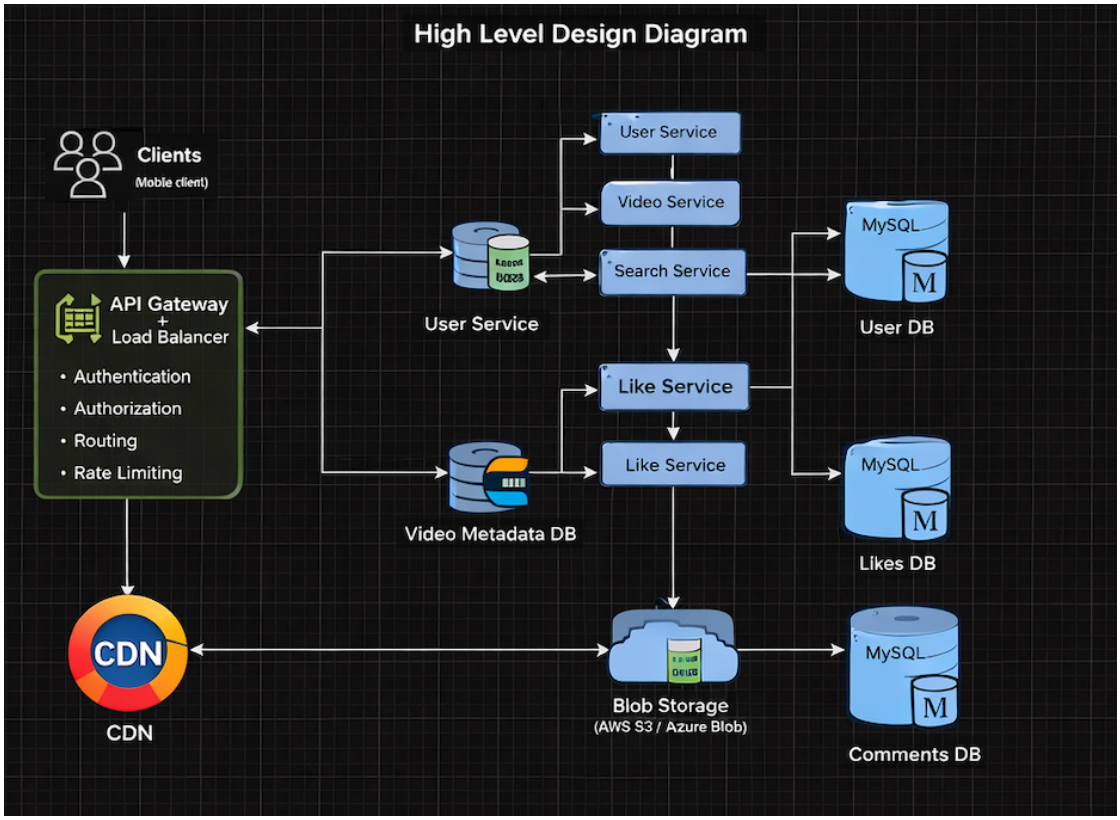
- 360p
- 720p
- 1080p
- 4K

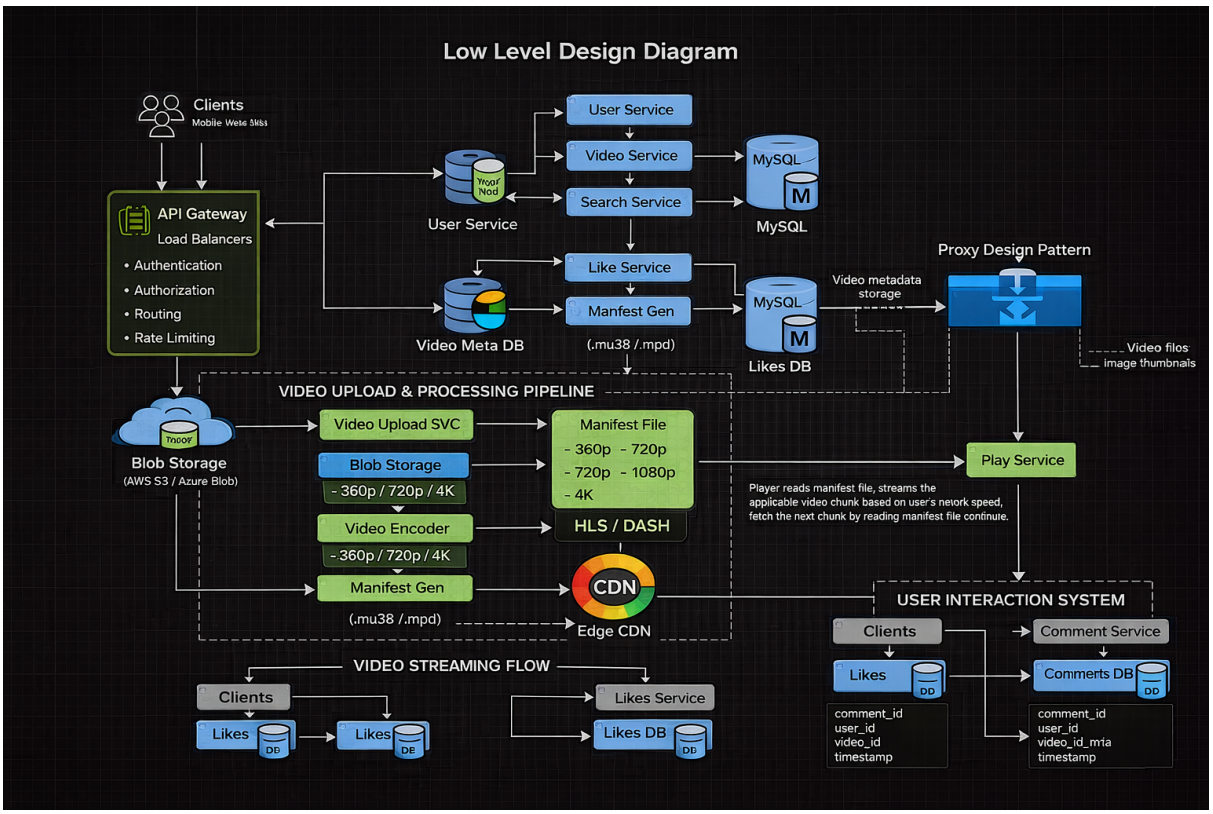
Step-4: **Manifest file (.m3u8 / .mpd)** is generated
Step-5: Manifest file is stored in storage
Step-6: CDN caches video chunks for faster streaming

7. Scalability Solution:

To support millions of users:

- Use CDN for global video delivery
- Use horizontal scaling of microservices
- Store videos in distributed object storage
- Cache frequently accessed data in Redis
- Use load balancers to distribute traffic
- Use stateless services for easy scaling
- Separate databases for users, likes, comments, and metadata





8. Learning Outcomes (What I Have Learnt):

- Learned how large scale video streaming systems like YouTube are designed.
- Understood functional and non-functional requirements of OTT platforms.
- Learned about video streaming technologies like HLS and DASH.
- Understood the role of CDN in reducing latency and buffering.
- Learned how microservices architecture improves scalability.
- Understood how likes, comments, and search features are implemented in distributed systems.